Java Exercise 3:
Bank Account with Advanced Concepts

Duration: 30 minutes

Instructions: Modify the previous bank account exercise where you are to implement Exception handling and lambda expression. Accomplish the exercise based on the details listed below.

Requirements:

- Define a new package exception where all the exception classes are created.
- Throw specific exceptions for various conditions (e.g., insufficient funds, invalid withdrawal or deposit amounts). Handle multiple exceptions such as insufficient funds, invalid amount, and account frozen.
- Use lambda expressions to filter and sort a list of transactions (e.g., sort all deposits, or filter transactions over a certain amount).
- All of the classes must have Java Documentation and are checked with Check Style.
- Before submission, your program should be tested thoroughly and is functional. You may refer to the next page for the test cases.

Class Design:

- Custom Exception Handling Class:
  - **`InsufficientFundsException`**: Thrown when a withdrawal exceeds the available balance.
    - Is a subclass of `Exception` class.
    - **Fields**:
      - `String message`: Holds the `message` passed from the constructor.
    - **Constructor**:
      - `InsufficientFundsException`(): With parameter `message`. It initializes the `message` using the parameter.
    - **Methods**:
      - `getMessage`(): returns the `message`.
  - **`InvalidAmountException`**: Thrown when an invalid amount (negative or zero) is deposited or withdrawn.
    - Is a subclass of `Exception` class.
    - **Fields**:
      - `String message`: Holds the `message` passed from the constructor.
    - **Constructor**:
      - `InvalidAmountException`(): With parameter `message`. It initializes the `message` using the parameter.
    - **Methods**:
      - `getMessage`(): returns the `message`.
    -
  - **`AccountFrozenException`**: Thrown when operations are performed on a frozen account.
    - Is a subclass of `Exception` class.
    - **Fields**:

        - `String message`: Holds the `message` passed from the constructor.
    - **Constructor**:
        - `AccountFrozenException()`: With parameter `message`. It initializes the `message` using the parameter.
    - **Methods**:
        - `getMessage()`: returns the `message`.
- Concrete Class of Interface: **BankAccountManager** (modifications)
    - **Methods**:
        - `List<Transaction> filterTransactionsAtOrAbove(double amount, List<Transaction> txList)`: Filter transactions above or equal to `amount` using lambda expression.
        - `List<Transaction> sortTransactionsByAmount(List<Transaction> txList)`: Sort transactions by amount (ascending order) using lambda expression.
- Class: **BankAccount** (modifications)
    - **Methods**:
        - `void deposit(double amount)`: Throw an:
            - invalid amount (message: `"Invalid amount. Must be greater than 0"`) and
            - account frozen exception (message: `"Account is FROZEN. Unable to transact."`)
        - `void withdraw(double amount)`: Throw an:
            - invalid amount (message: `"Invalid amount. Must be greater than 0"`),
            - insufficient amount (message: ,`"Insufficient balance."`) and
            - account frozen exceptions (message: `"Account is FROZEN. Unable to transact."`).

Test Case	Description	Operations	Expected Output
1	`Deposit with negative amount`	`Call deposit(-500).`	`Amount deposited: -500.00` `Invalid amount. Must be greater than 0`
2	`Deposit when account is frozen`	`1. Call freezeAccount().` `2. Call deposit(11500).`	`Account is FROZEN. Unable to transact.`
3	`Withdraw with negative amount`	`Call withdraw(-100).`	`Amount withdrawn: -100.00` `Invalid amount. Must be greater than 0`

4	Withdraw with insufficient funds	Call withdraw(1500).	Amount withdrawn: 1500.00 Insufficient balance. Current balance: 500.13
5	Withdraw when account is frozen	1. Call freezeAccount(). Call withdraw(500).	Account is FROZEN. Unable to transact.
6	Filter transaction at or above a certain amount.	Call filterTransactionsAtOrAbove(200, acc1.getTransactionHistory()).	Filtering transactions at or above 200... [Transaction [type=Deposit, amount=1000.125, time_stamp=2026-06-11 10:43] , Transaction [type=Withdraw, amount=500.0, time_stamp=2026-06-11 10:43]]
7	Sort transaction by amount.The top most has the lowest amount value.	Call sortTransactionsByAmount(acc1.getTransactionHistory()).	Sorting transactions by amount... [Transaction [type=Withdraw, amount=100.0, time_stamp=2026-06-11 10:43] , Transaction [type=Withdraw, amount=500.0, time_stamp=2026-06-11 10:43] , Transaction [type=Deposit, amount=1000.125, time_stamp=2026-06-11 10:43]]